```
<?php ob_start(); ?>
<!DOCTYPE html>
<?php $mainPara = '<p>consectetur adipiscing elit</p>' ?>
<?php header('Content-type: text/html; charset=utf-8'); ?>
<html>
  <head>
    <meta charset="utf-8" />
    <title>The easiest page you'll ever create</title>
  </head>
<body>
  <h1>Lorem ipsum dolor sit amet</h1>
  <?= $mainPara ?>
  </body>
</html>
```

Now, that is actually a pretty neat trick; you can just write your markup and only have logic and variable data injected where it's needed. This is what templating is: text with placeholders that have their values filled dynamically to create the final output. PHP has multiple (excellent) HTML templating libraries available, but it is in fact also its own templating language. So why am I recommending that you don't use native PHP templating? The answer lies in something we touched on in Chapter 10, "Programming With Objects": *separation of concerns*.

## The Problem

The power of having access to the entire range of functions and features of PHP while also assembling markup for output introduces a potential problem: you can do literally *anything*. Need a quick database fetch? No problem, just shove `<?php $pdoConnection->query('SELECT ...'); ?>` where it's needed. The trouble is that you now have to keep track of potentially time-consuming calls *and* assemble a response. With a complex page, you might choose to break your markup into multiple files and only fetch the ones you need with a `require()` or `include()` statement. Now imagine that one of those included files has a DB read call in there, and it's being included in a loop structure… all of a sudden, you're making multiple DB reads when only one was sufficient.